

コマ 12: struct / typedef / . / ->

今日のゴール

`typedef struct { ... } Name;`、構造体変数、`.`、`->` を実装する。

第 10 回までに、型サイズと配列・ポインタを扱えるようになった。この回では、複数のフィールドをまとめた構造体を扱う。

```
typedef struct {  
    int x;  
    int y;  
} Point;
```

`Point` は、`x` と `y` という 2 つのフィールドを持つ型である。

1 この回で扱う範囲

対象にする機能は次の通り。

種類	例
typedef struct	<code>typedef struct { int x; int y; } Point;</code>
直接メンバアクセス	<code>p.x</code>
ポインタ経由メンバアクセス	<code>p->x</code>
構造体ポインタ引数	<code>int f(Point *p)</code>

この回では、構造体のフィールドは `int`、`char`、ポインタ程度に限定する。構造体のネスト、構造体代入、グローバル構造体変数は扱わない。

2 AST を確認する: .

まず、`.` を使うプログラムがどのような AST になるか確認する。

```
python3 scaffold/parse_viewer.py sessions/12_struct_typedef/tests/dot_access.c
```

このプログラムの内容は次の通り。

```
typedef struct {
    int x;
    int y;
} Point;

int main() {
    Point p;
    p.x = 3;
    p.y = 4;
    return p.x + p.y;
}
```

実行すると、次のような S 式が表示される。

```
(program
 (funcdef "main" :type int (params)
  (block
   (decl "p" :type (type "Point"))
   (exprstmt
    (assign
     (member "." "x" (var "p"))
     (num 3)))
   (exprstmt
    (assign
     (member "." "y" (var "p"))
     (num 4)))
   (return
    (add
     (member "." "x" (var "p"))
     (member "." "y" (var "p"))))))))
```

`p.x` は AST 上では `(member "." "x" (var "p"))` になる。

3 AST を確認する: ->

次に、構造体ポインタを使うプログラムを確認する。

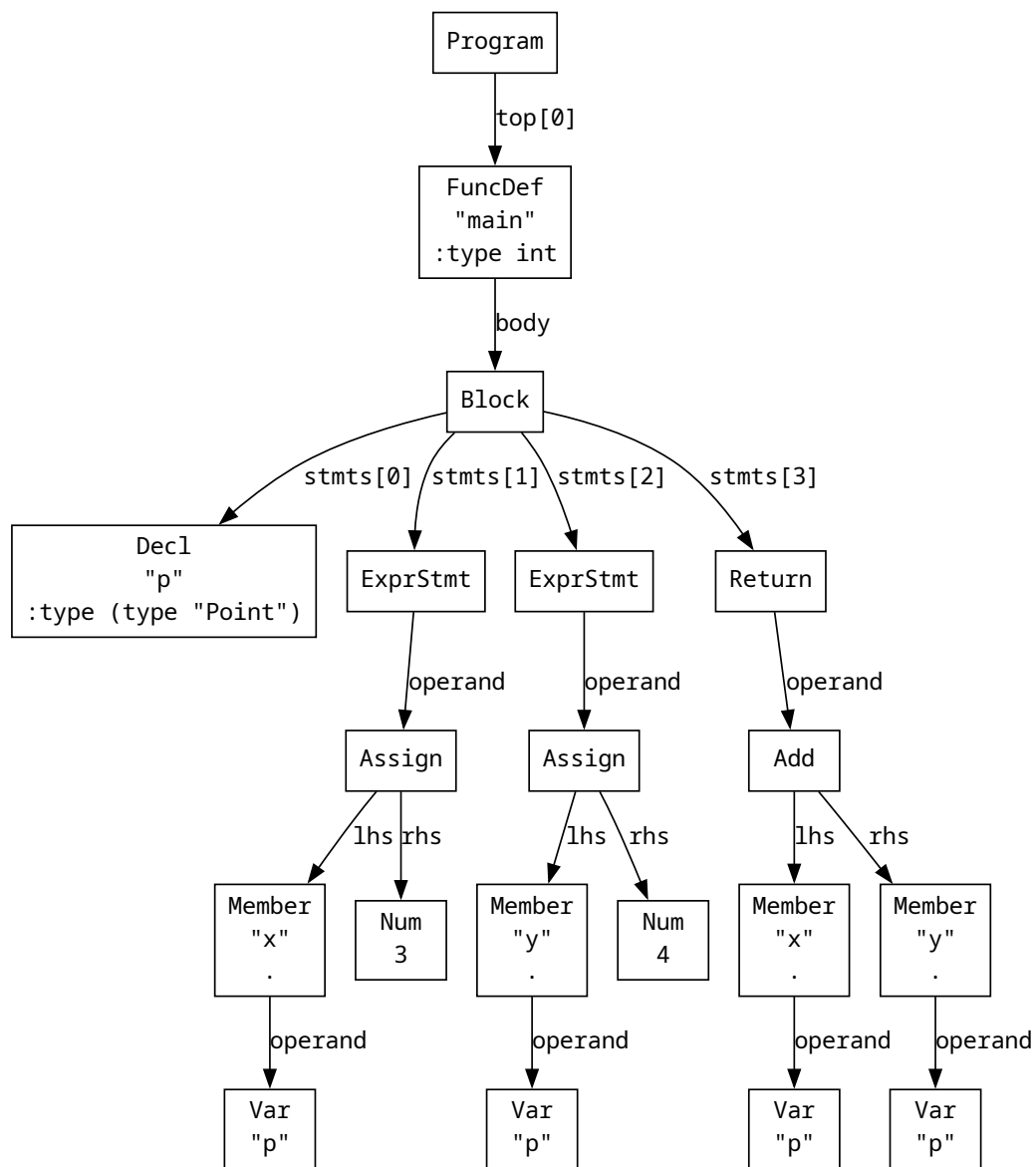


図1 dot_access.c のAST

```
python3 scaffold/parse_viewer.py sessions/12_struct_typedef/tests/arrow_access.c
```

このプログラムの内容は次の通り。

```
typedef struct {
    int x;
    int y;
} Point;
```

```

int distance_sq(Point *p) {
    return p->x * p->x + p->y * p->y;
}

int main() {
    Point p;
    p.x = 3;
    p.y = 4;
    return distance_sq(&p);
}

```

実行すると、次のような S 式が表示される。

```

(program
  (funcdef "distance_sq" :type int
    (params
      (param "p" :type
        (ptr (type "Point")))))
    (block
      (return
        (add
          (mul
            (member "->" "x" (var "p"))
            (member "->" "x" (var "p")))
          (mul
            (member "->" "y" (var "p"))
            (member "->" "y" (var "p"))))))))
  (funcdef "main" :type int (params)
    (block
      (decl "p" :type (type "Point"))
      (exprstmt
        (assign
          (member "." "x" (var "p"))
          (num 3)))
      (exprstmt
        (assign
          (member "." "y" (var "p"))
          (num 4)))
      (return
        (call "distance_sq"
          (args
            (addr (var "p"))))))))

```

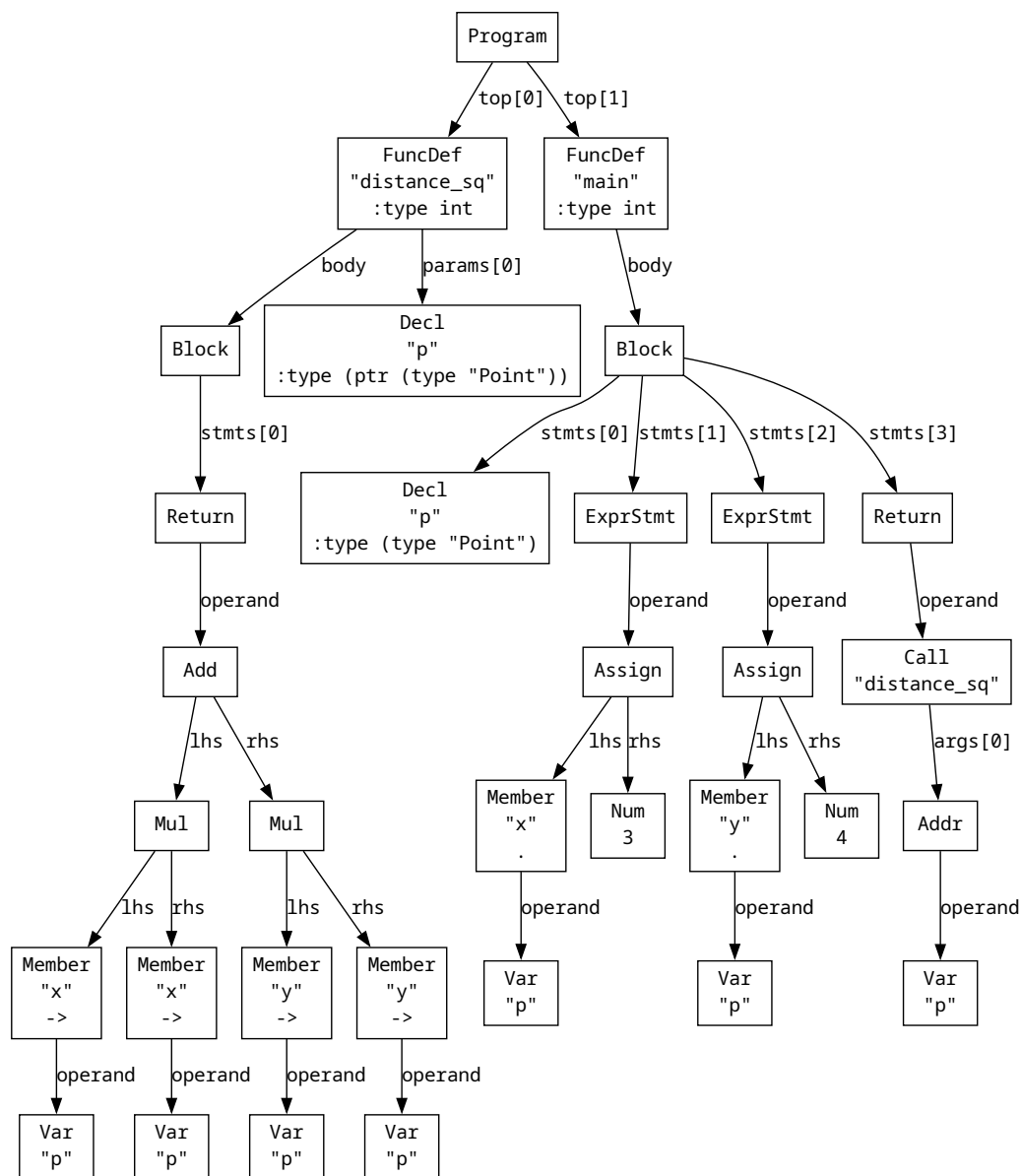


図2 arrow_access.c のAST

`p->x` はAST上では(member `"->"` `"x"` (var `"p"`))になる。意味としては`(*p).x`と考えればよい。

4 構造体のメモリレイアウト

`Point` は `int x; int y;` を持つ。この講義では`int` を4バイトとして扱うため、次のように配置する。

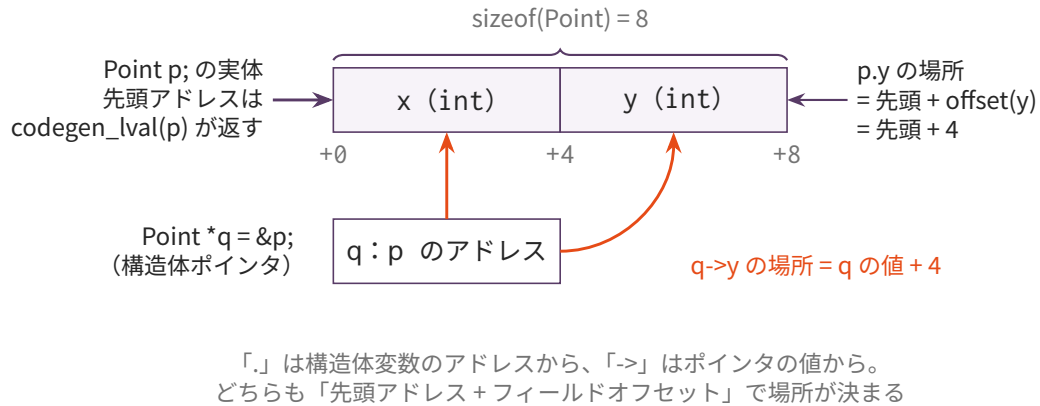


図 3 Point のメモリ配置と . / -> のアドレス計算

フィールド	オフセット	サイズ
x	0	4
y	4	4

したがって Point 全体のサイズは 8 バイトである。

```
Point p;

p + 0 byte : x
p + 4 byte : y
```

図の q は `Point *q = &p;` としたポインタである (`distance_sq(&p)` の仮引数も同じ状態になる)。`.` は構造体変数のアドレスから、`->` はポインタの値から、どちらも「+ フィールドオフセット」で場所が決まる。

5 typedef struct を読む

Parser は `typedef struct { ... } Point;` のフィールド一覧を AST には残さない。そのため、この回ではソース文字列を走査して構造体情報を集める。

```
# スケルトンがあらかじめ source から構造体情報を収集し、
# self._struct_defs としてコンストラクタに渡す設計
```

`self._struct_defs` は構造体名をキーとする辞書である。たとえば `Point` のエントリは次の形をとる。

```
{
    "size": 8,
    "fields": {
        "x": {"offset": 0, "size": 4},
        "y": {"offset": 4, "size": 4},
    },
}
```

スケルトンは次のクラスメソッドでソースから構造体情報を抽出する。

```
@classmethod
def register_typedef_names(cls, source: str) -> set[str]:
    """typedef 名を収集する"""
    ...

@classmethod
def parse_struct_defs(cls, source: str) -> dict[str, dict]:
    """構造体定義をパースしてフィールド情報を返す"""
    ...
```

これらの関数は、次の形だけを対象にすればよい。

```
typedef struct {
    int x;
    int y;
} Point;
```

6 . のコード生成

`p.x` の `lvalue` は、`p` のアドレスにフィールドオフセットを足したアドレスである。

```
address(p.x) = address(p) + offset(x)
```

したがって `codegen_lval()` では次のようにする。

```

if node.kind == ND_MEMBER and not node.is_arrow:
    self.codegen_lval(node.operand)
    ty_str = self._type_of_lval(node.operand)
    field = self._struct_defs[ty_str]["fields"][node.name]
    self.emit(f"    addi a0, a0, {field['offset']}")
    return

```

7 -> のコード生成

`p->x` は `(*p).x` と同じ意味である。`p` の値は構造体先頭アドレスなので、まず `p` を `rvalue` として評価する。

```
address(p->x) = value(p) + offset(x)
```

```

if node.kind == ND_MEMBER and node.is_arrow:
    self.codegen(node.operand)
    ptr_ty = self._type_of_expr(node.operand)           # e.g. "Point *"
    strut_ty = self._deref_ptr(ptr_ty)                  # "Point"
    field = self._struct_defs[strut_ty]["fields"][node.name]
    self.emit(f"    addi a0, a0, {field['offset']}")
    return

```

8 実装手順

1. スケルトンの `importlib` 継承により第 11 回の `Codegen` クラスを引き継ぐ（あらかじめ書かれている）
2. `CodegenNN.register_typedef_names(source)` で `typedef` 名を収集する
3. `CodegenNN.parse_struct_defs(source)` で構造体定義をパースし `self._struct_defs` に渡す
4. `self._type_of_lval()` で `typedef` 名→`ty_str` を解決し、`ND_MEMBER` を追加する
5. `self.codegen_lval()` に `ND_MEMBER` ハンドラを追加する（`self._struct_defs` からフィールドオフセットを引く）
6. `load()` / `store()` はフィールド型に応じて既存のものを使う

9 テスト

```
python3 scaffold/test_runner.py sessions/12_struct_typedef
```

この回の主要テストは次の通り。

テスト	内容	期待値
dot_access.c	p.x, p.y の読み書き	7
arrow_access.c	Point *p に対する p->x	25